CS3230 Lecture 09 – Graph

Steven Halim
(stevenhalim@gmail.com)

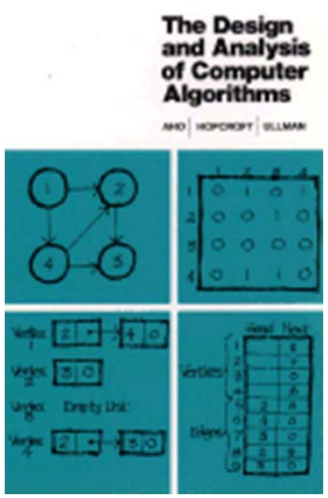
CS3230 Lecture 09 - Graph

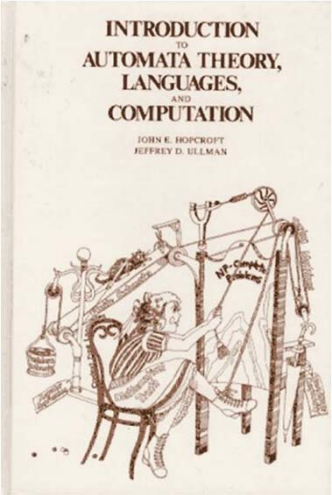
Lecture Topics and Readings

- Very quick review on Graph Data Structure and BFS traversal (CLRS, Chapter 22, Section 22.1-22.2, CP3 Section 2.4.1+4.2)
- Deeper review on DFS traversal (CLRS, Sec 22.3; CP3 Sec 4.2)
- Connected Component (Undirected graph) + random graph
- P versus NPC versions of related graph problems
  - Shortest path versus Longest path
  - Euler tour versus Hamiltonian tour
  - 2-CNF-SAT versus 3-CNF-SAT
- Strongly Connected Component (Directed graph, CLRS, Section 22.5; CP3 Section 4.2.9+S9.17)

John Hopcroft

- PhD in Stanford, Work in Princeton,
  Finally based in Cornell
- 1986 Turing award (with Robert Tarjan)
  - For his contribution in algorithm and data-structure
- 2010 IEEE John von Neumann Medal (with Jeffrey Ullman)
  - Laying the foundations for the fields of automata and language theory
- He has three famous books





S. Rao Kosaraju

- Sambasiva Rao Kosaraju is a professor of Computer Science at Johns Hopkins University
  - He is an Indian
- We will learn his algorithm for finding Strongly Connected Components in a Directed Graph today

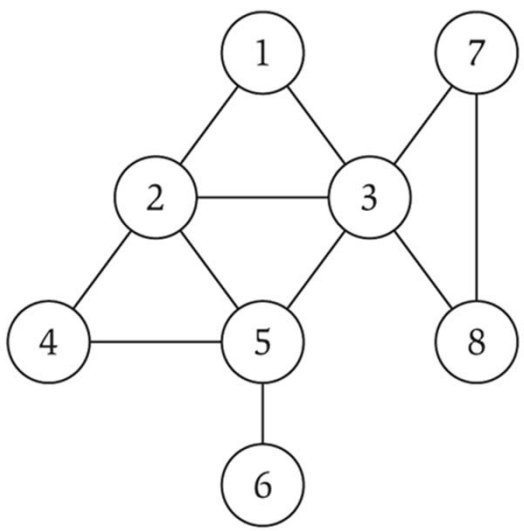
Review of Graph Algorithms

Just a VERY Quick Review

Undirected Graphs

Undirected graph G = (V, E)

- V = vertices
- E = edges between pairs of vertices
  - Captures pairwise relationship between vertices
- Graph size parameters: n = |V|, m = |E|



V = { 1, 2, 3, 4, 5, 6, 7, 8 }

E = { 1-2, 1-3, 2-3, 2-4, 2-5, 3-5, 3-7, 3-8,
4-5, 5-6, 7-8 }

n = 8

m = 11

9-11 Terrorist Network

Social network graph

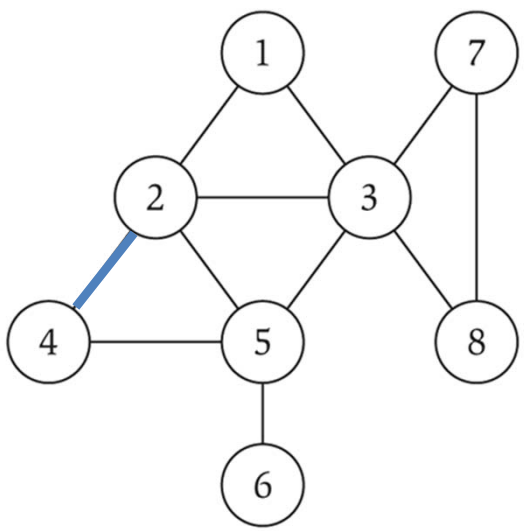
- Vertex: people
- Edge: relationship between two people

Reference: Valdis Krebs, http://www.firstmonday.org/issues/issue7_4/krebs

Graph DS: Adjacency Matrix

Adj Matrix: n-by-n matrix with $A_{uv} = 1$ if (u, v) is an edge

- Two representations of each edge
- Space proportional to n^2
- Checking if (u, v) is an edge takes $\Theta(1)$ time
- Identifying all edges takes $\Theta(n^2)$ time



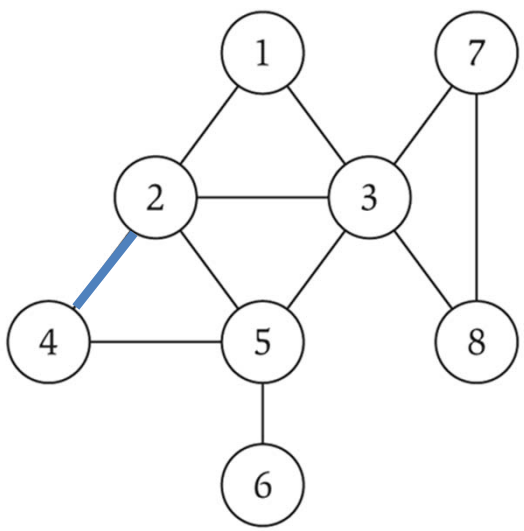
	1	2	3	4	5	6	7	8
1	0	1	1	0	0	0	0	0
2	1	0	1	1	1	0	0	0
3	1	1	0	0	1	0	1	1
4	0	1	0	1	1	0	0	0
5	0	1	1	1	0	1	0	0
6	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	1
8	0	0	1	0	0	0	1	0

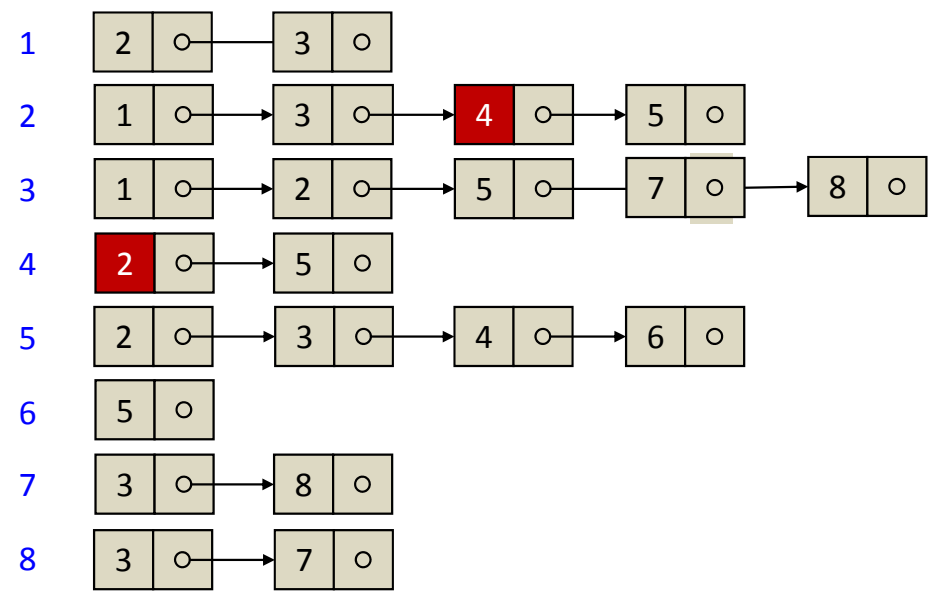
Graph DS: Adjacency List

Adjacency List: Array of vertex lists, one for each vertex

- Two representations of each edge
- Space proportional to m + n
- Checking if (u, v) is an edge takes O(deg(u)) time
- Identifying all edges takes Θ(m + n) time

degree = number of neighbors of u

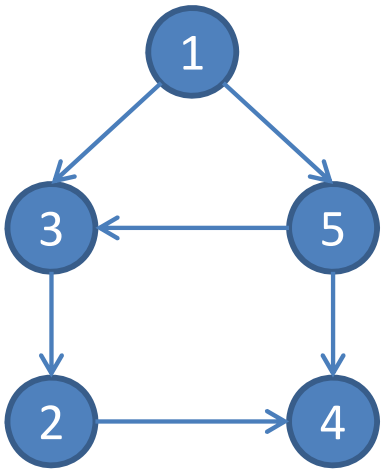




Directed Graph

Directed graph: G = (V, E)

- V = vertices
- E = directed edges between pairs of vertices
- Graph size parameters: n = |V|, m = |E|

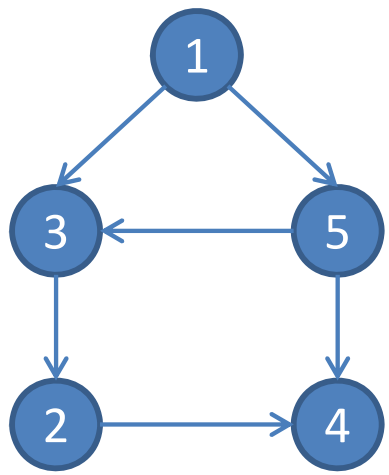


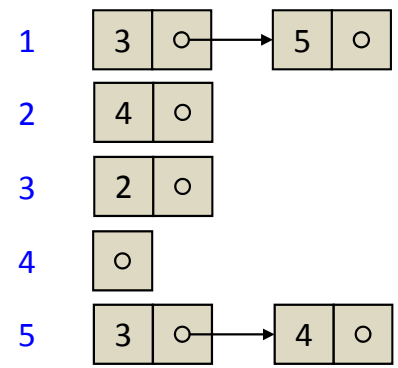
V = {1, 2, 3, 4, 5}
E = { (1, 3), (1, 5), (2, 4), (3, 2), (5, 3), (5, 4) }
n = 5
m = 6

Graph Representation

	1	2	3	4	5
1	0	0	1	0	1
2	0	0	0	1	0
3	0	1	0	0	0
4	0	0	0	0	0
5	0	0	1	1	0

Adjacency Matrix





Adjacency List

"Infinite" number of MORE examples at:
http://www.comp.nus.edu.sg/~stevenha/visualization/graphds.html

Graph Traversal: BFS

Another Quick Review

Graph Traversal

Graph traversal is a way to visit all vertices in a graph

There are two common methods for traversal:

1. **Breadth-First Search** (only re-mentioned briefly today, please refer to CS2010/CS2020 material, CLRS Sec 22.2, or CP3 Section 4.2)
2. **Depth-First Search** (discussed today, <u>we will expand our understanding</u> on this traversal technique, reference: CLRS Sec 22.3+22.5, CP3 Sec 4.2)

Breadth-First Search (BFS)

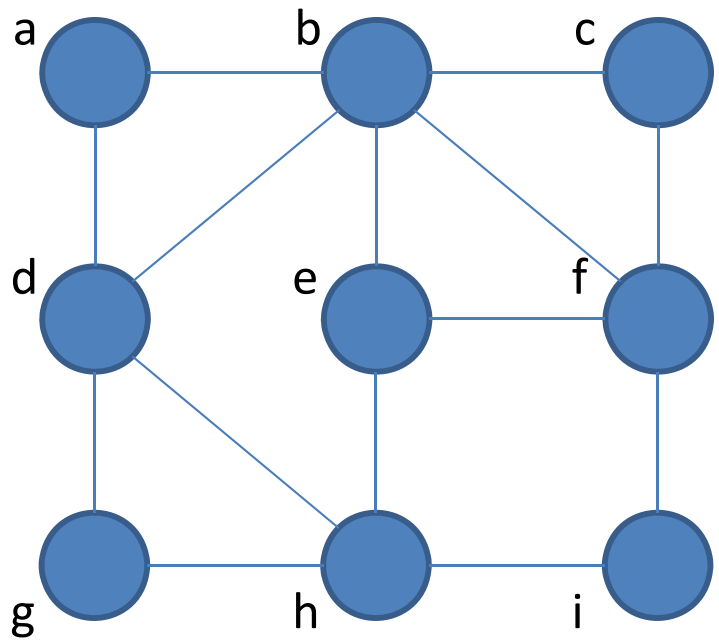
Starting from any vertex s (called source), Breadth-First Search visits the vertices in increasing distance from s

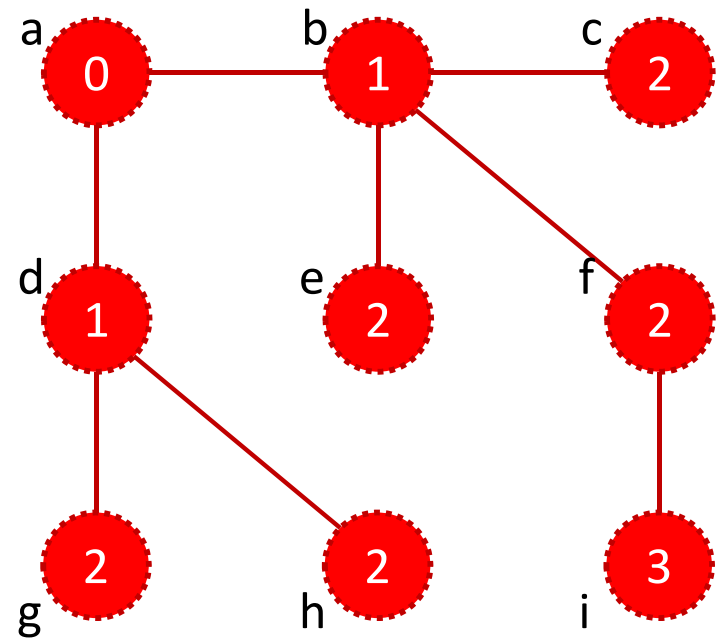
Breadth-First Search generates two information:

- The shortest distance from s to each reachable vertex
- A breadth-first search tree with root s such that the path from any reachable vertex v to s is a shortest path (in terms of the number of edges used)

Example

Assume vertex a is the source





Breadth-First Search tree
(Number inside the vertex
is the shortest path value)

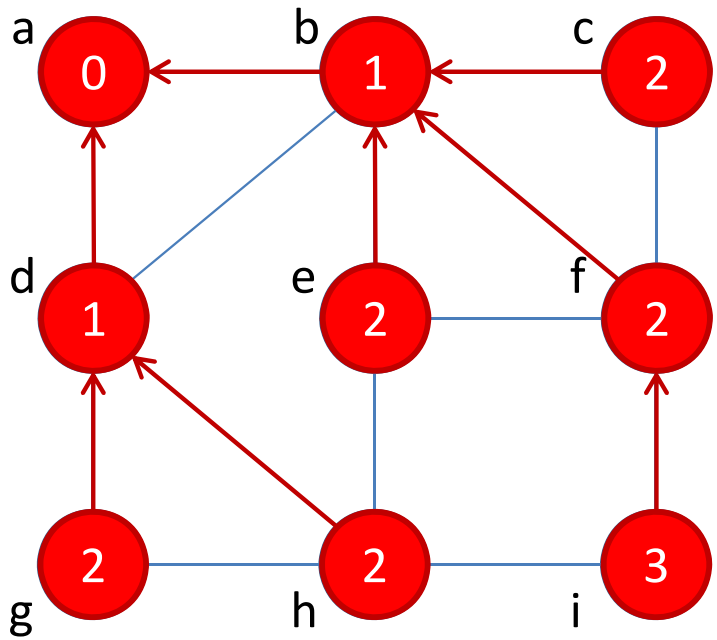
Breadth-First Search algorithm

BFS(G, s)

```
set Q = (s);
set d(s) = 0;
while Q is not empty
    u = dequeue(Q);
    set visited[u] = true;
    for each neighbor v of u
        if visited[v] = false, then
            set π(v) = u; // π is the predecessor/parent array
            set d(v) = d(u)+1;
            enqueue(Q, v);
```

Example

Assume vertex a is the source and
neighbors are ordered based on increasing vertex label



Q=(a)
Q=(b,d)
Q=(d,c,e,f)
Q=(c,e,f,g,h)
Q=(e,f,g,h)
Q=(f,g,h)
Q=(g,h,i)
Q=(h,i)
Q=(i)
Q=Φ

"Infinite" number of MORE examples at:
http://www.comp.nus.edu.sg/~stevenha/visualization/dfsbfs.html

BFS Running Time Analysis

We only enqueue each vertex once

- In total, we visit |V| vertices

For each vertex, we will scan each of its neighbor once

- In total, we scan |E| neighbors
  - Note for undirected graph, |E| = the number of <u>directed</u> edges = 2 * the number of <u>undirected</u> edges

The running time of Breadth-First Search is O(|V|+|E|)

Graph Traversal: DFS

With Deeper Details, literally…

Depth-First Search (DFS)

Unlike BFS, DFS tries to search <u>as deep as possible</u>

It will reports a Depth-First Search tree

- This DFS tree, as we will see soon, is quite useful ☺

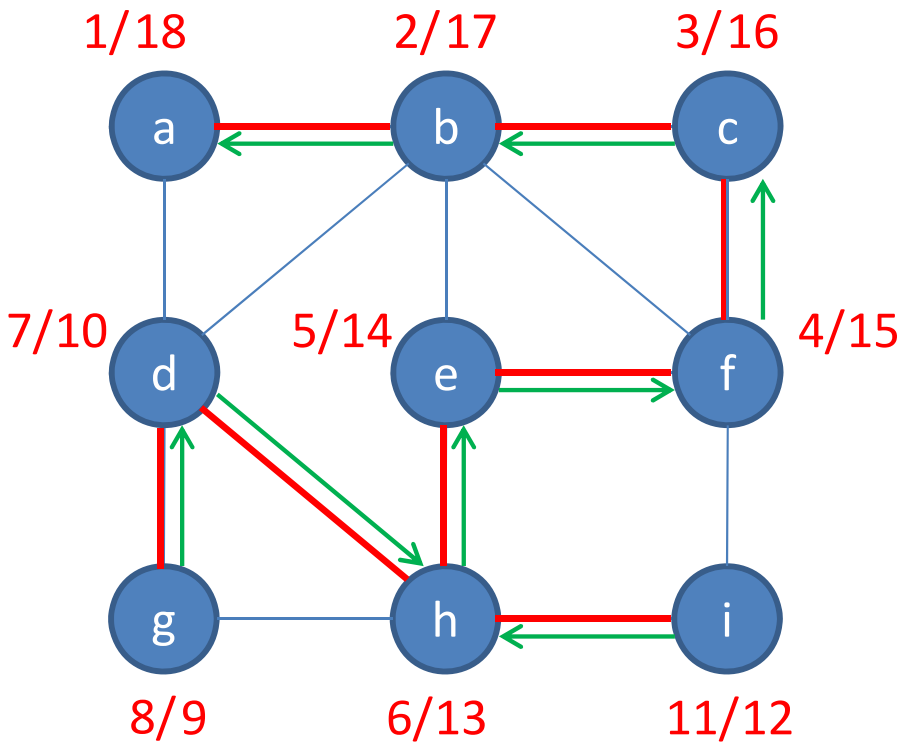
Depth-First Search algorithm

```
time = 0; // global variable

DFS-VISIT(G, u)
    time = time+1;
    d(u) = time; // record the discovery time of u
    for each neighbor v of u
        if visited[v] = false
            π(v) = u;
            DFS-VISIT(G, v);
    time = time+1;
    f(u) = time; // record the finishing time of u
```

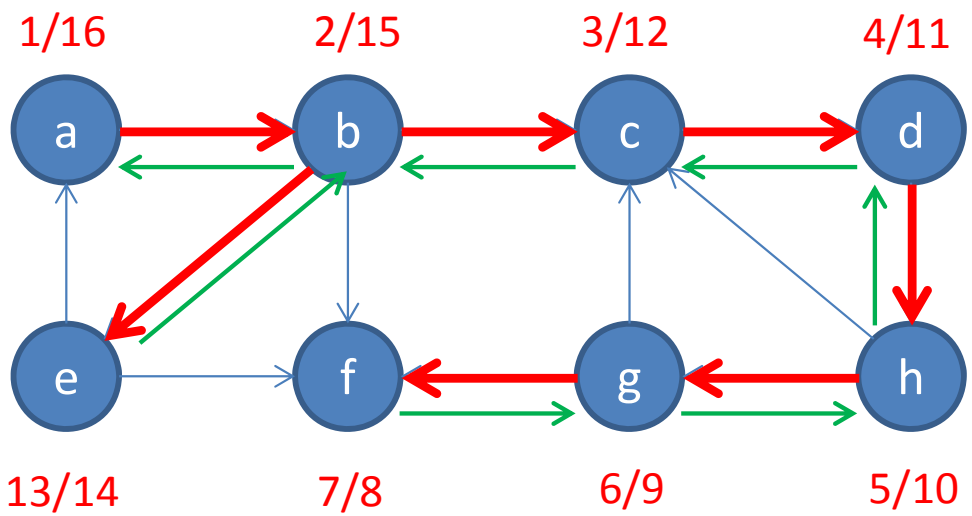
Example DFS on Undirected graph

Assume vertex a is the source and
neighbors are ordered based on increasing vertex label



Example DFS on Directed graph

Assume vertex a is the source and
neighbors are ordered based on increasing vertex label



"Infinite" number of MORE examples at:
http://www.comp.nus.edu.sg/~stevenha/visualization/dfsbfs.html

DFS Running Time Analysis

DFS visits every node once

For each node, DFS visits every edge once

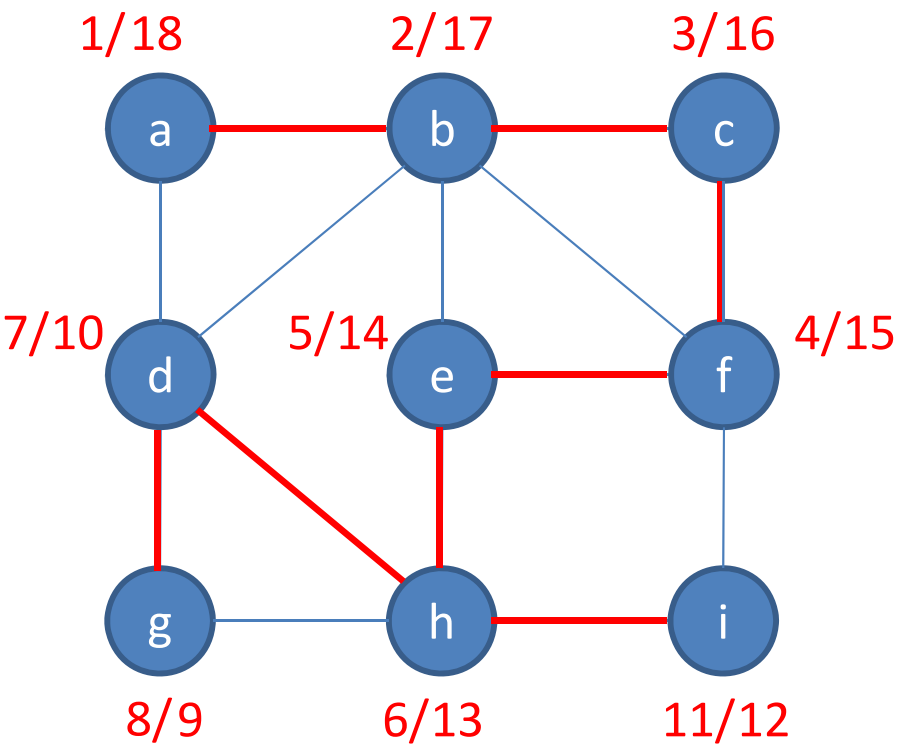
Sounds familiar?

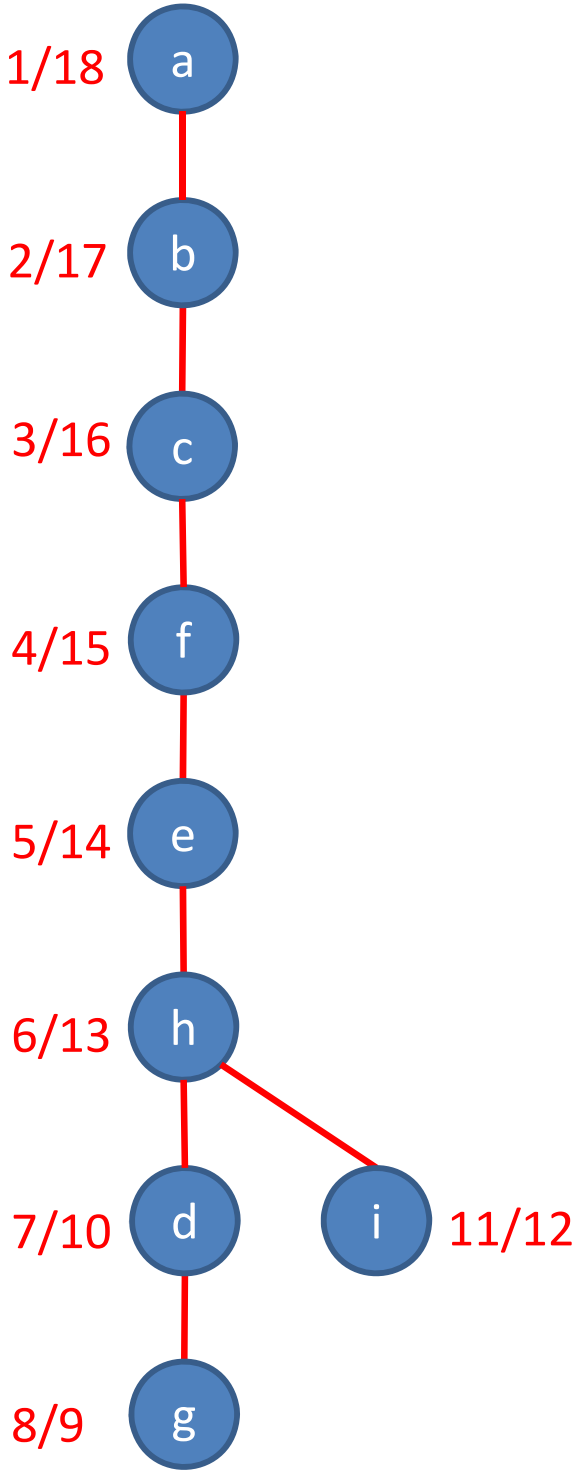
- Yes this is just like BFS

Running time: $O(|V|+|E|)$

DFS Tree Property 1

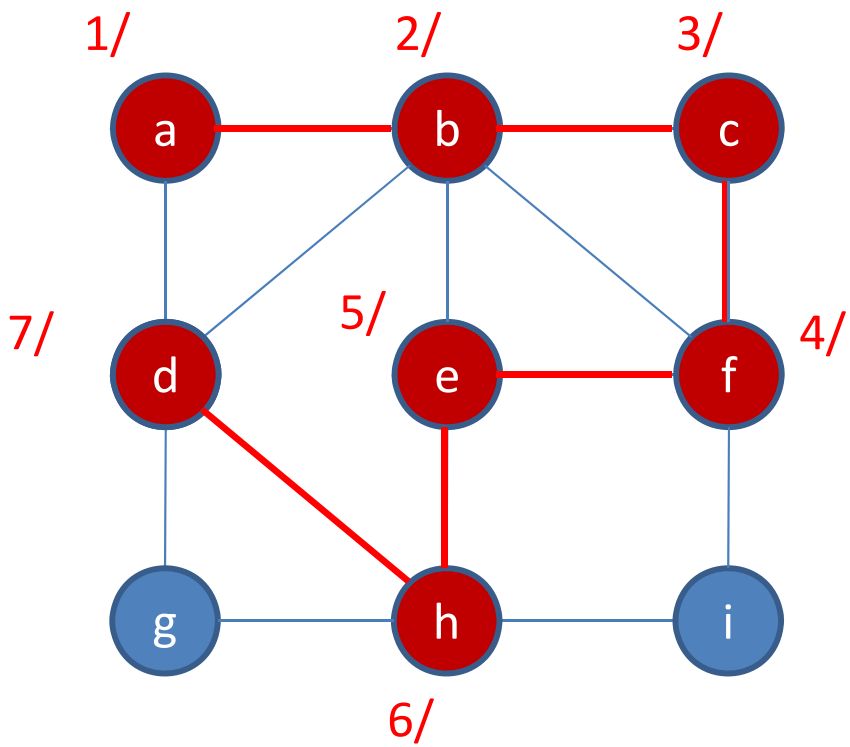
For any u and v,
(d(u), f(u)) and (d(v), f(v)) are either nested or do not overlap

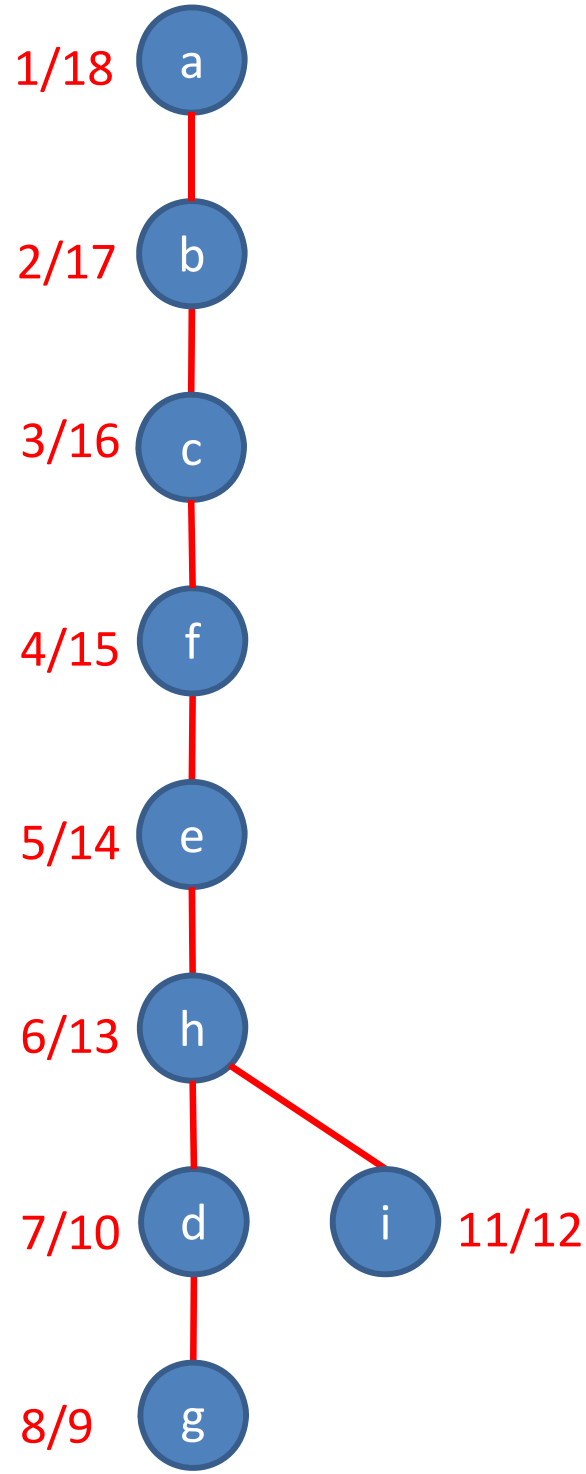




DFS Tree Property 2

When DFS(G, u) is called, all unvisited vertices v reachable from u are descendants of v in the DFS tree





Edge Classification

Given a DFS tree for G,
the edges in G can be classified as follows:

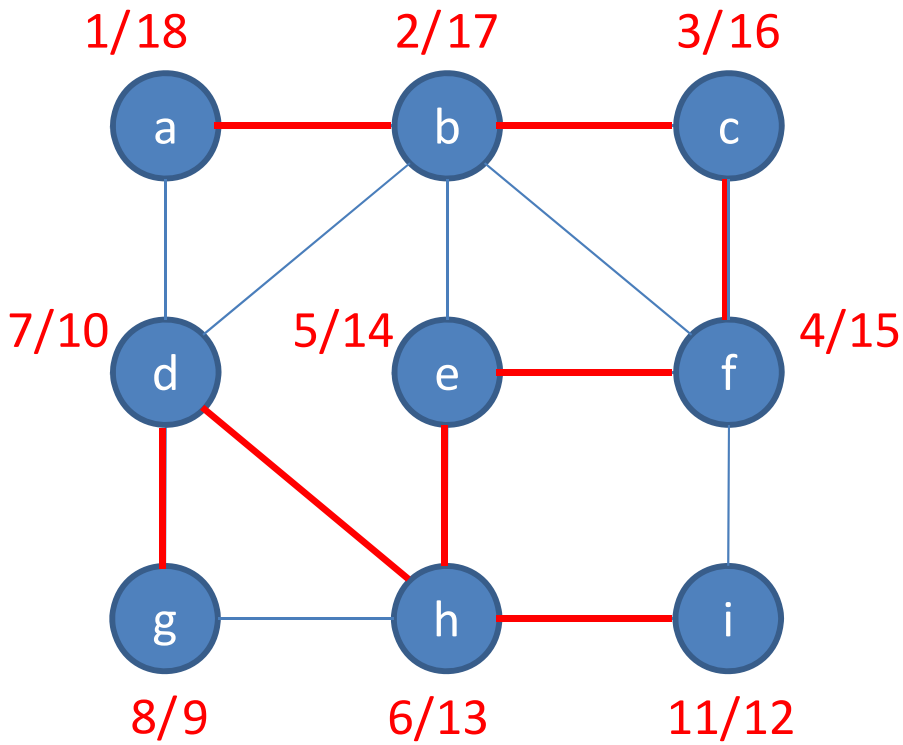
1. **Tree edges**: simply the edges on the DFS tree
2. **Back edges**: non-tree edges connecting a vertex to its ancestor

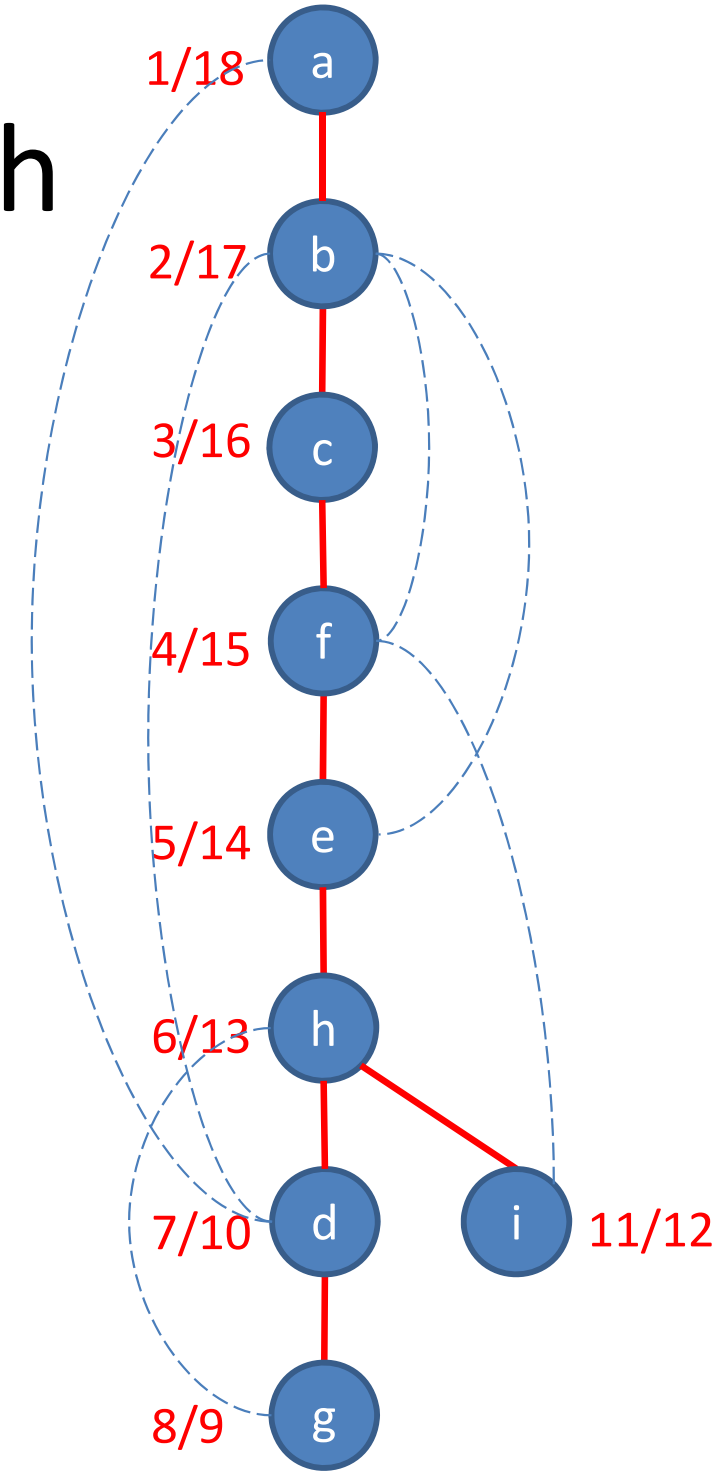
(for directed graph, we have two more types:)

3. **Forward edges**: non-tree edges connecting a vertex to its descendant
4. **Cross edges**: other edges

DFS on <u>undirected</u> graph

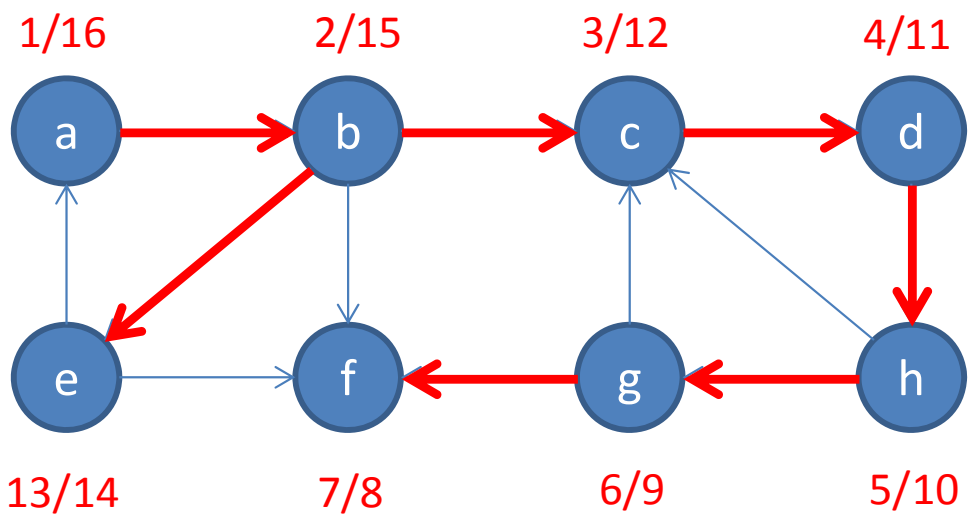
Only tree (red) and
back (blue-dashed) edges

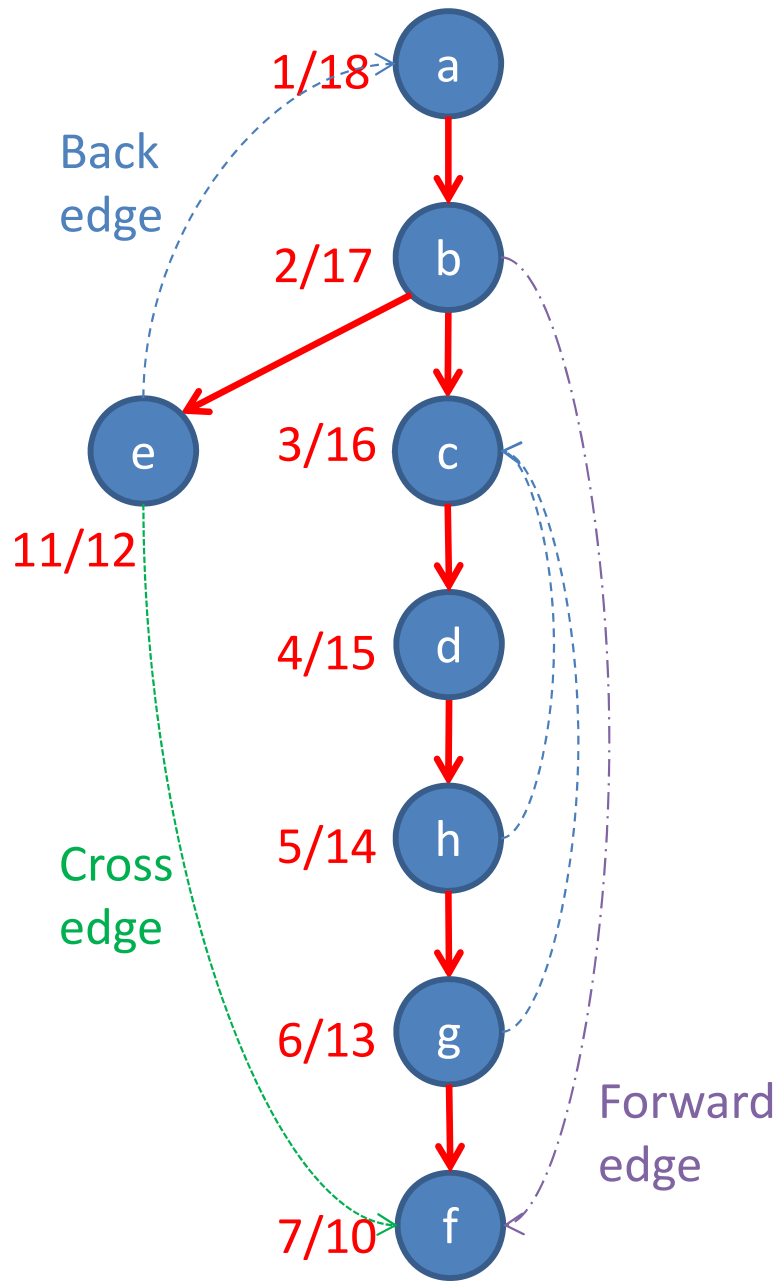




DFS on directed graph

All four types appear: tree (red), back (blue-dashed), forward (purple-dash-dotted), and cross (green-dotted) edges





Applications of Depth-First Search

1. Depth-First Search is good to detect cycle (by detecting the back edges)
2. For finding (one) topological sort of a DAG
3. For checking if the graph is Biconnected
   - Or to find cut vertices or bridges otherwise
   - PS: Another name for cut vertex is articulation point
4. For finding Strongly Connected Component
   - This will be our focus today ☺

Graph Decomposition

CC (and SCC, later)

Graph Decomposition

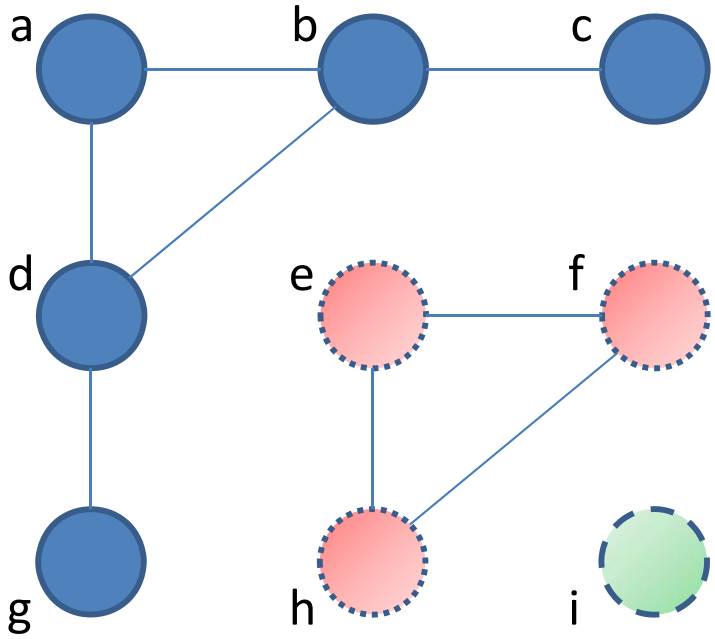
Graph decomposition is a basic skill to reduce a big problem into a number of small problems

Today, we discuss two cases:

- Finding Connected Component in undirected graph
  - With side story on Random Graph (NOT EXAMINABLE)
- Finding Strongly Connected Component in directed graph
  - This will be very useful for your PA2 Subtask EF

Connected Component (CC)

A connected component of an <u>undirected graph</u> is a maximal connected subgraph



There are 3 CCs:

{a, b, c, d, g}

{e, f, h}

{i}

Algorithm to Find CCs

```
for every vertex u in G
   if visited(u) = false
      DFS(G, u)
      report the nodes in the DFS tree of u as a component
```

This algorithm runs in O(|V|+|E|) time although we call DFS(G, u) several times

- Same argument, each vertex and each edge is visited once

Note: We can replace DFS(G, u) by BFS(G, u)

Side Story: CCs in Random Graph (1)

Let G(**n**, ***p***) be a random graph with **n** vertices formed by randomly connecting two vertices with probability ***p***

The <u>expected</u> number of edges of G(**n**, ***p***) is $\binom{n}{2}p$

- The <u>expected</u> degree is **n*p***

Side Story: CCs in Random Graph (2)

Property (Erdős–Rényi model, <u>no need to memorize</u>):

- If **np** < 1, then **G(n, *p*)** will almost surely have no connected components of size larger than **O(log n)**
- If **np** = 1, then **G(n, *p*)** will almost surely have a largest component whose size is of order $\boldsymbol{n^{2/3}}$
- If **np→c > 1**, where **c** is a constant, then **G(n, *p*)** *will almost surely* (that is, this is just a probability) have a unique **giant component** containing a positive fraction of the vertices and all other components are of size **O(log n)**

PS: Property of random graphs should be interesting for preparing test-cases in programming contests

Example

Consider the friendship network (e.g. Facebook)

Assume each person has 10 friends in average

- Then, np → 10

By the property of random graph,
it is almost sure that there is a giant component

- In other word, most of us are “friends of friends”

Small-World Effect

For a connected graph **G**, let the diameter **d(G)** to be the maximum distance between any two vertices in **G**

$\mathbf{d(G) \approx O(\log_c n)}$ where $\mathbf{c = np}$

The proportion of **random graphs** for which $\mathbf{d(G) > c \log n}$ trends to 0 when **n** is large

References

For more information on random graph, read:

- http://www.santafe.edu/media/workingpapers/02-02-005.pdf
- F. Chung and L. Lu “Diameter of Random Graphs”
- Or Google around…

Graph Problems in P vs NPC

We detour back to the topic of NP-Completeness before continuing with the topic of SCC

Comparison (the “obvious” ones)

P

1. DFS/BFS variants
2. MST: Prim’s/Kruskal’s
3. SSSP: Dijkstra’s/ Bellman Ford’s
4. APSP: Floyd Warshall’s

(From CS2010/2020/3230)

NP-Complete

1. CLIQUE
2. VERTEX-COVER
3. TRAVELING-SALESMAN -PROBLEM (TSP)

(From Lecture08 proofs)

Comparison (the "subtle" ones)

P

1. Shortest Path
2. Eulerian Path
   (discussion in tutorial)
3. 2-CNF-SAT
   (L8, PA2 EF)

NP-Complete

1. Longest Path
   (discussion in tutorial)
2. TSP (basically Hamiltonian Path)
3. 3-CNF-SAT
   (L8, PA2 ABCD)

3 versus 2-CNF-SAT

3-CNF-SAT is **NP-Complete** (proof in L8)

2-CNF-SAT, however, is in **P**

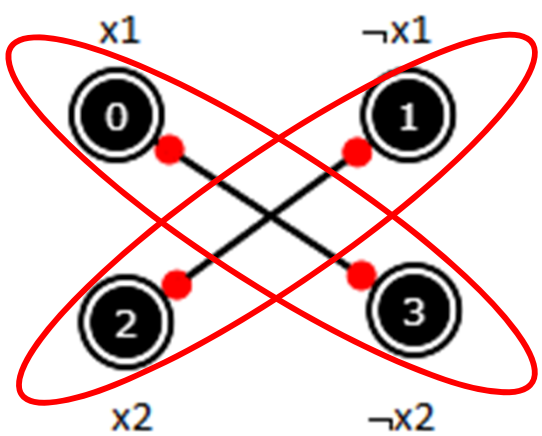
- We can reduce 2-CNF-SAT into a problem of finding SCCs of the “implication graph” and then check if and only if there is no variable that belongs to the same SCC as its negation
- Finding SCCs of a directed graph will later be shown to be solvable in O(V+E), i.e. polynomial time
- Example in the next slide

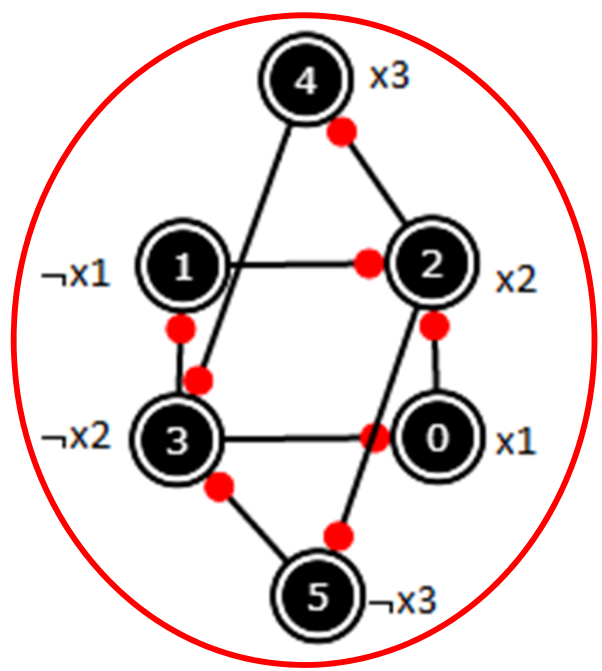
2-CNF-SAT $\leq_p$ Checking SCCs of the Implication Graph

(a v b) is the same as ¬a → b and ¬b → a

$\phi_1 = (x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2)$ and
$\phi_2 = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$
are equivalent to the following two implication graphs:

SATISFIABLE





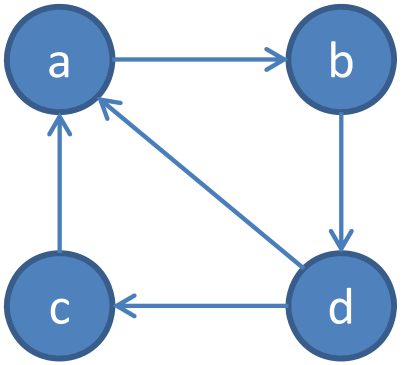
NOT SATISFIABLE

Strongly Connected Component (SCC)

The main takeaway from today
(you need it to solve PA2 subtask EF)

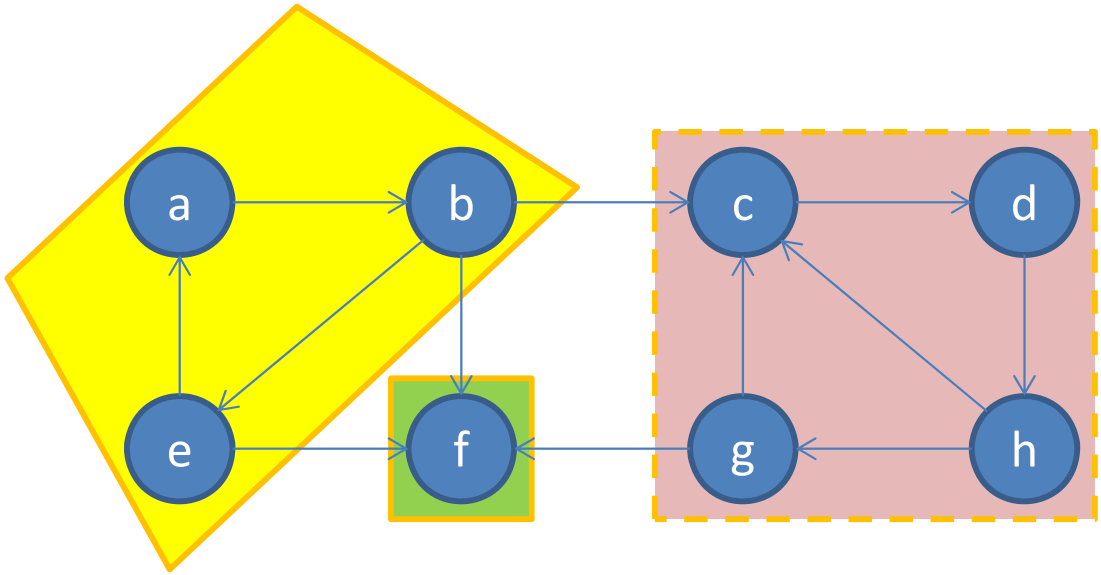
Definition of SCC

A directed graph G is strongly connected if there is a directed path from v_i to v_j <u>and also</u> from v_j to v_i for every v_i and v_j in G

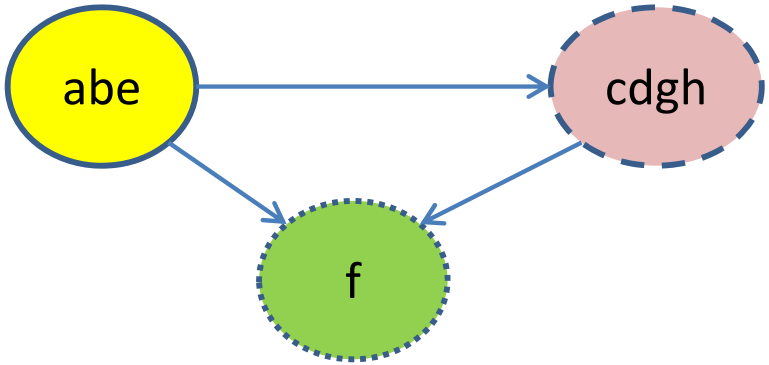


A Strongly Connected Component (SCC) is a maximal subgraph that is strongly connected

Example



Three SCCs:
{a, b, e}
{f}
{c, d, g, h}



After contracting all edges within each SCCs, we obtain a Directed Acyclic Graph (DAG)

This is the **component graph**

Application of SCCs

We can use SCCs to partition the graph

- It is used in compiler analysis, data mining, etc
- It is often used in parallel computing

Typical strategy:

1. Find the SCCs
2. Solve the problem for each component
3. Merge the result

Simple (but Slow) Solution

For every vertex u,
perform DFS to find all vertices reachable from u

For every two vertices u and v,
if u→ v <u>and</u> v→u, then put them into the same SCC

Running time = $O(|V| * (|V|+|E|))$

- PS: $|E|$ is $O(|V|^2)$

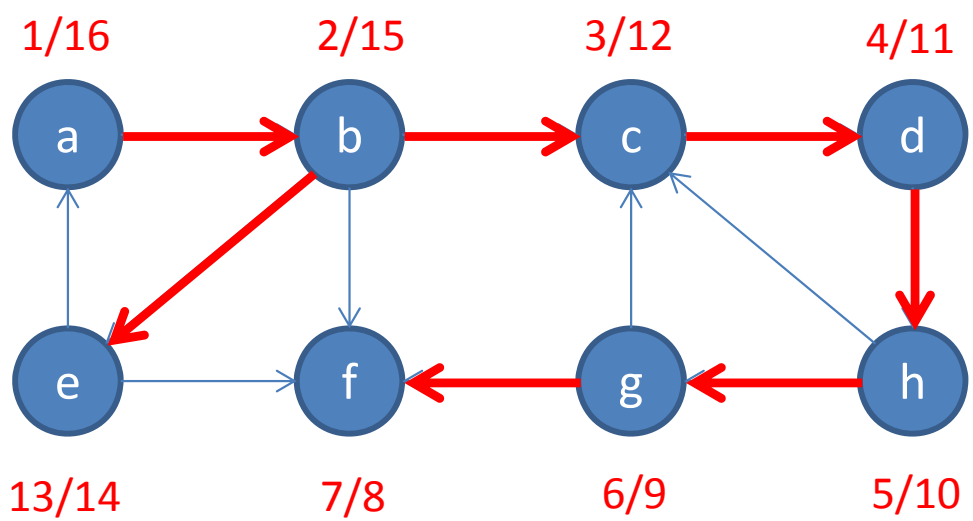
Can we do better?

Use Discovery and Finishing Time

Consider a set of vertices U

- Denote $f(U) = \max_{u \in U}\{ f(u) \}$
- Denote $d(U) = \min_{u \in U}\{ d(u) \}$

e.g. $f(\{b, c, d\}) = 15$ and $d(\{b,c,d\}) = 2$

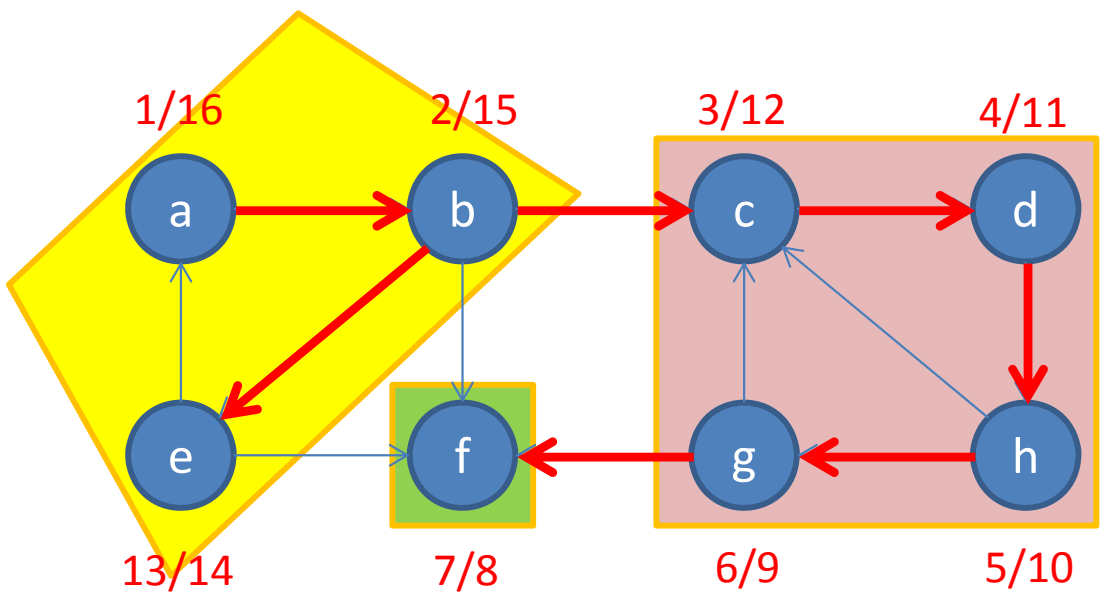


Observation 1: On Finishing Time

Consider two SCCs C and C’ in G

If there exists an edge (u, v) where $u \in C$ and $v \in C'$, then, $f(C) > f(C')$

- Because DFS has to finish exploring SCC C’ before returning to SCC C (backtrack via edge (u, v))



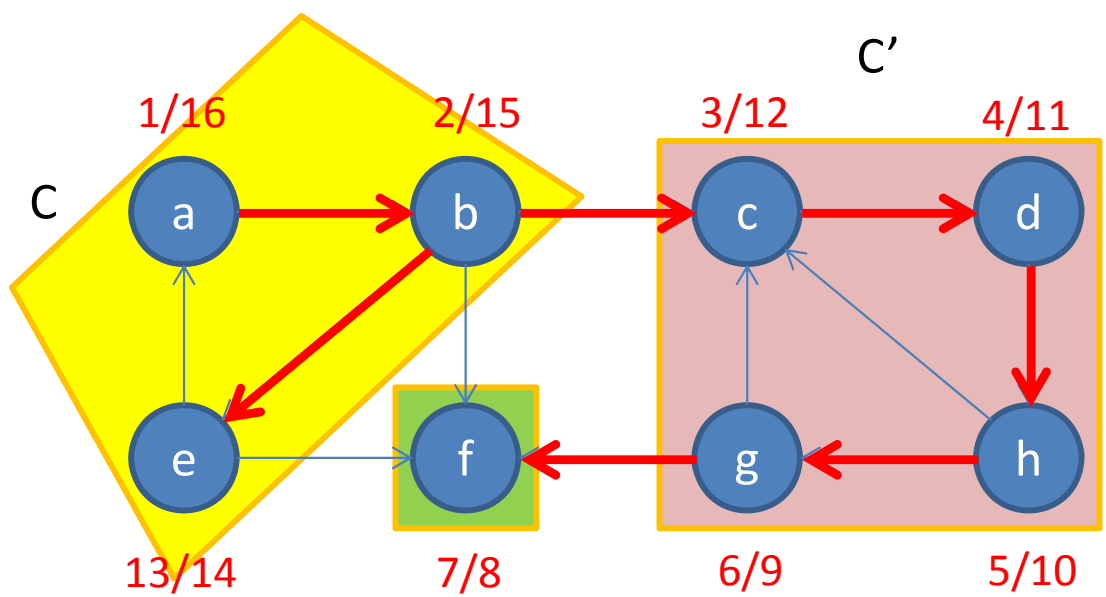
$f(\{a,b,e\}) = 16$
$f(\{c,d,g,h\}) = 12$
$f(\{f\}) = 8$

Proof

Two cases: $d(C) < d(C')$ or $d(C) > d(C')$

If $d(C) < d(C')$,

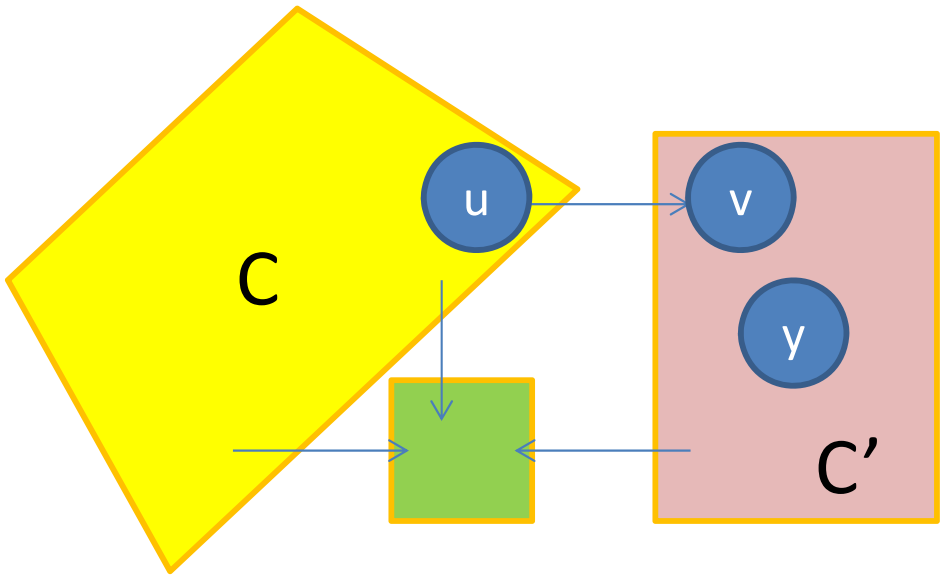
- Let x be the vertex such that $d(x) = d(C)$
- In DFS tree of G, all vertices in C' are descendants of x
- Hence, $f(x) > f(C')$
- Thus, $f(C) \geq f(x) > f(C')$



Proof

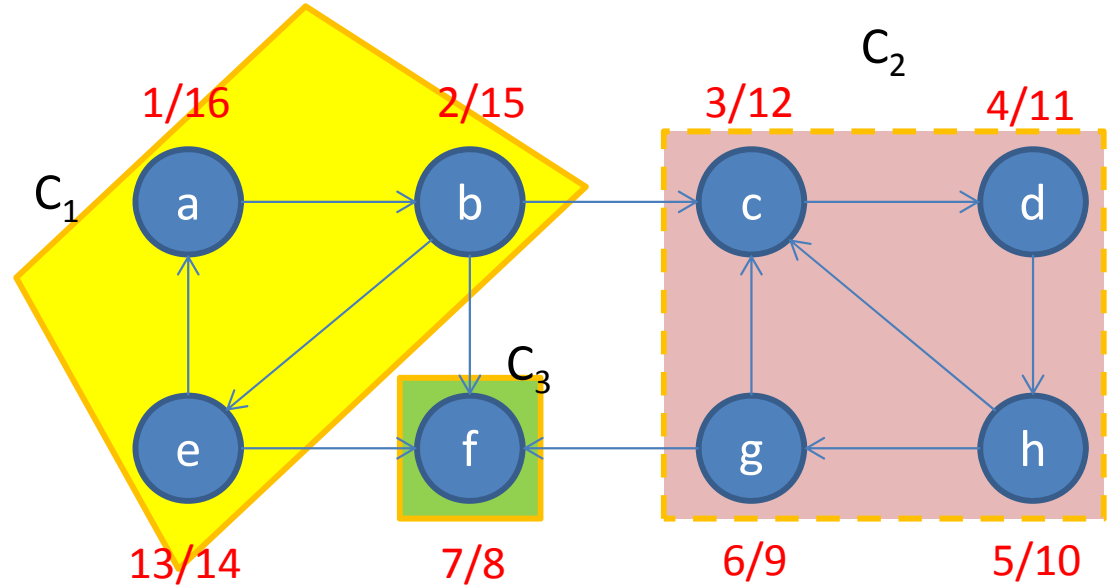
If d(C) > d(C’),

- Let y be the vertex such that d(y) = d(C’)
- Since edge (u, v) exists – see the previous two slides,
  by definition of SCC, all vertices in C are not reachable from y
- Hence, at the time f(C’), none of the vertices of C is visited
- Hence, f(C) > f(C’)



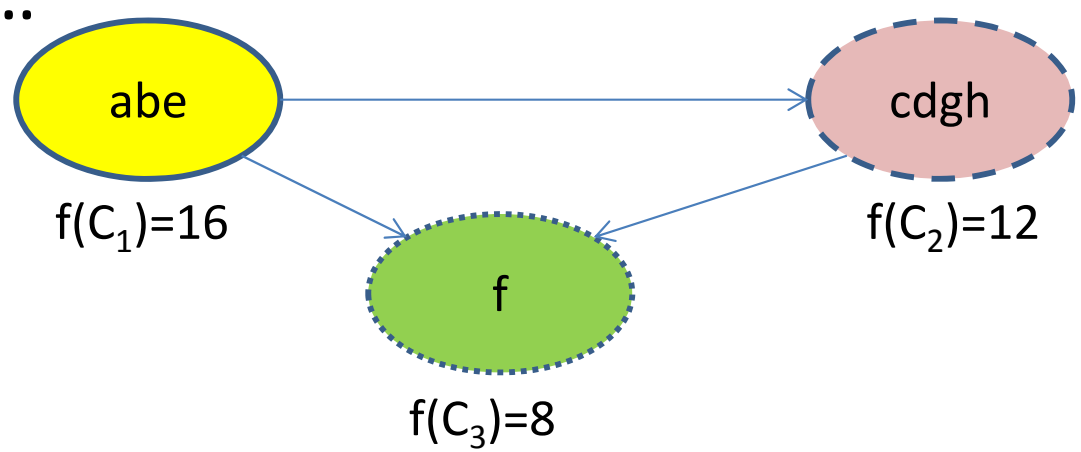
Topological Order of the SCCs

The finishing time gives the topological order of the SCCs in the component DAG



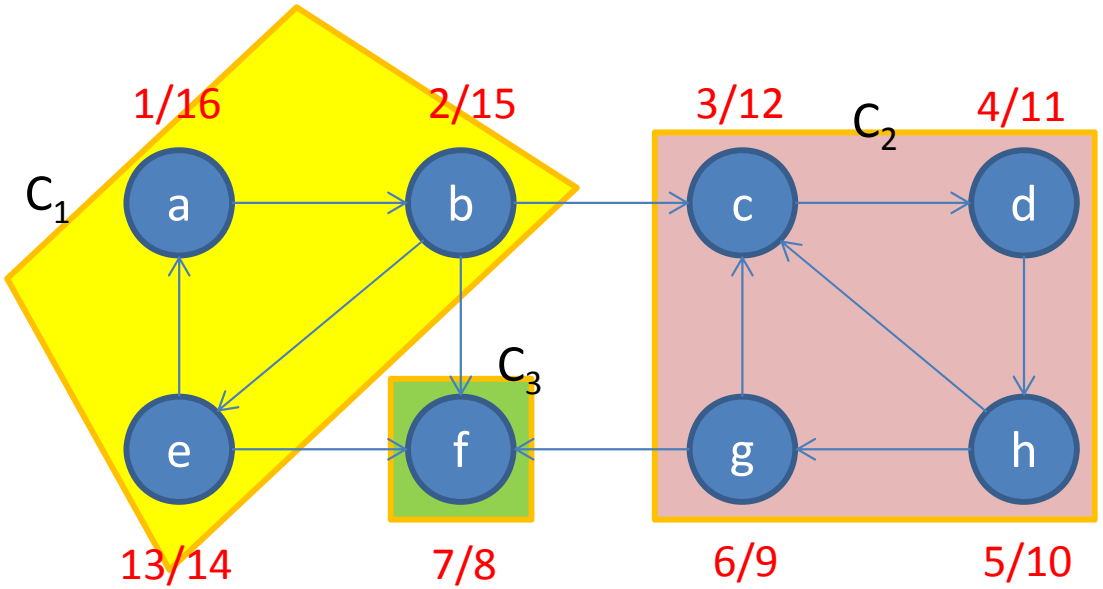
G

Kosaraju's algorithm tries to find the SCCs C_1, C_2, C_3, … in <u>decreasing order of f(), but in another graph G^T</u>



Observation 2:
G and G^T have the same SCCs!

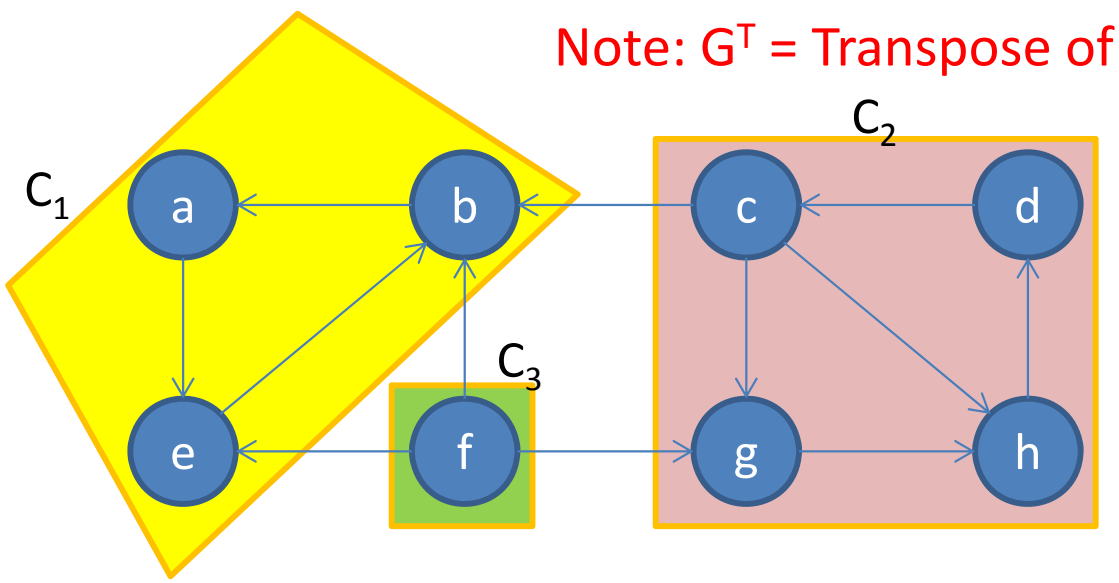
G



Three SCCs:
{a, b, e}
{f}
{c, d, g, h}

G^T

Note: G^T = Transpose of G



Three SCCs:
{a, b, e}
{f}
{c, d, g, h}

Now f(C) < f(C')

Kosaraju's Algorithm for Finding SCCs

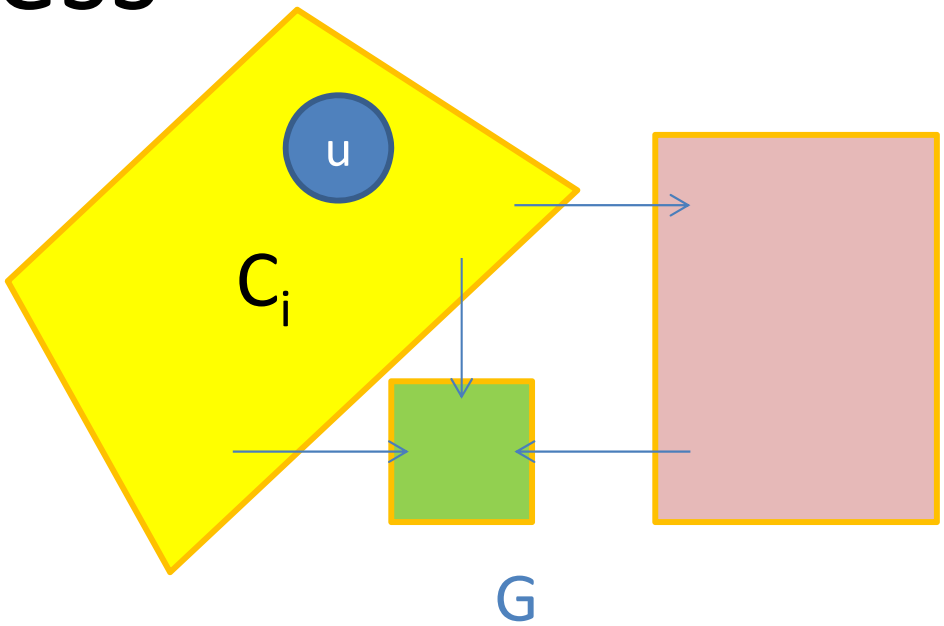
Input: A directed graph G

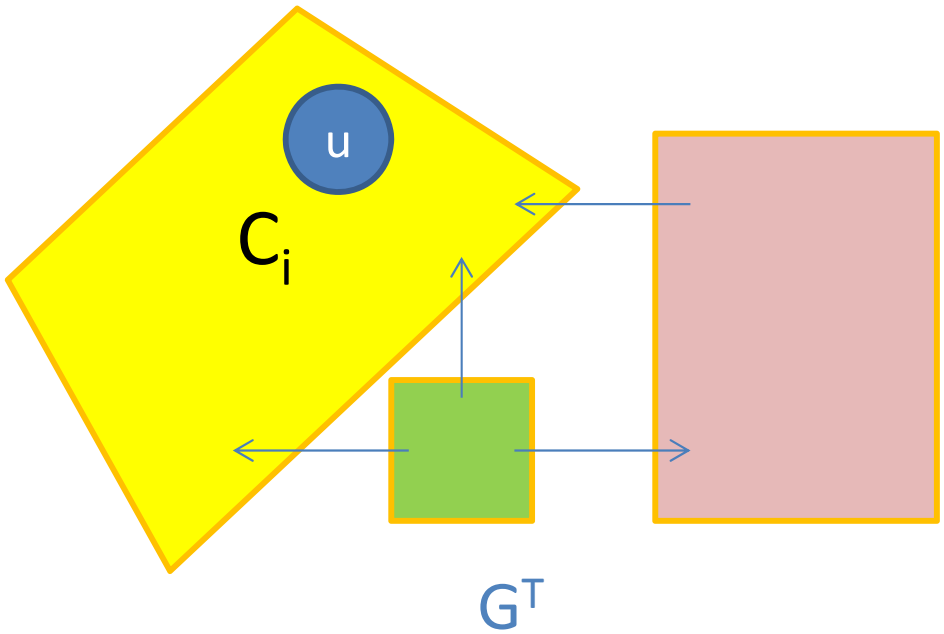
1. Call DFS(G)
   Find the finishing time f(u) for all vertices u in G
2. Compute G^T
   (note: this can be done together with input reading)
3. Call DFS(G^T)
   [Consider the vertices in order of decreasing f(u)]
4. Each DFS tree corresponds to one SCC ☺

Kosaraju's Correctness

Suppose we found SCCs in this order: $C_1, C_2, \ldots, C_{i-1}$

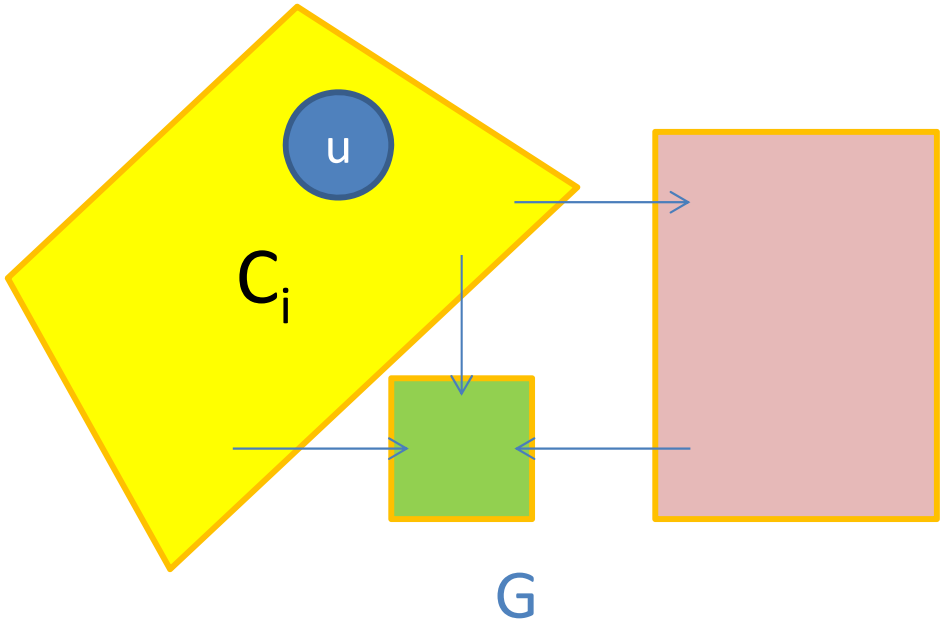
Lemma: When we perform $DFS(G^T, u)$ for an unvisited vertex u with the biggest f(u), the DFS tree corresponds to the SCC C_i

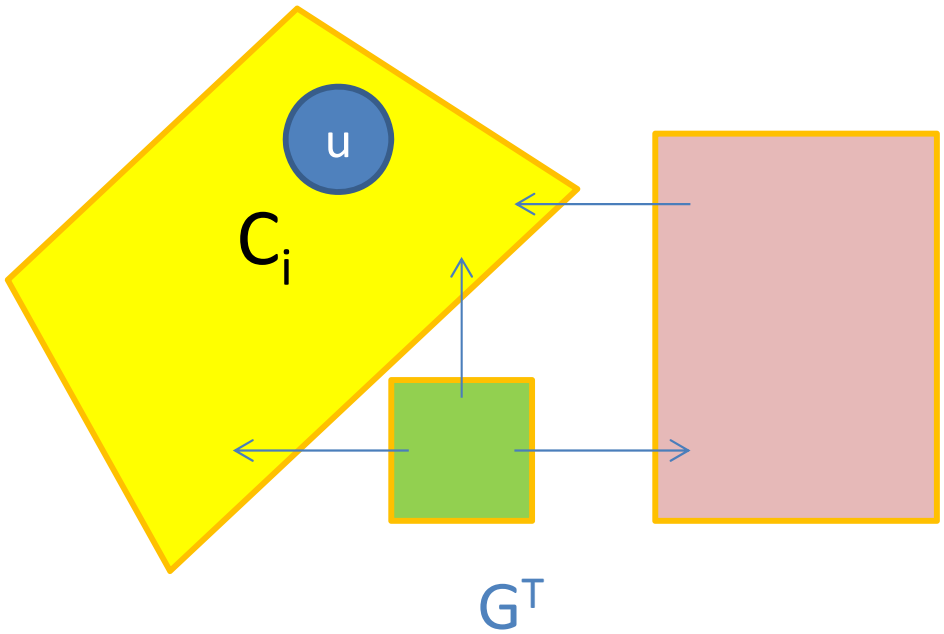




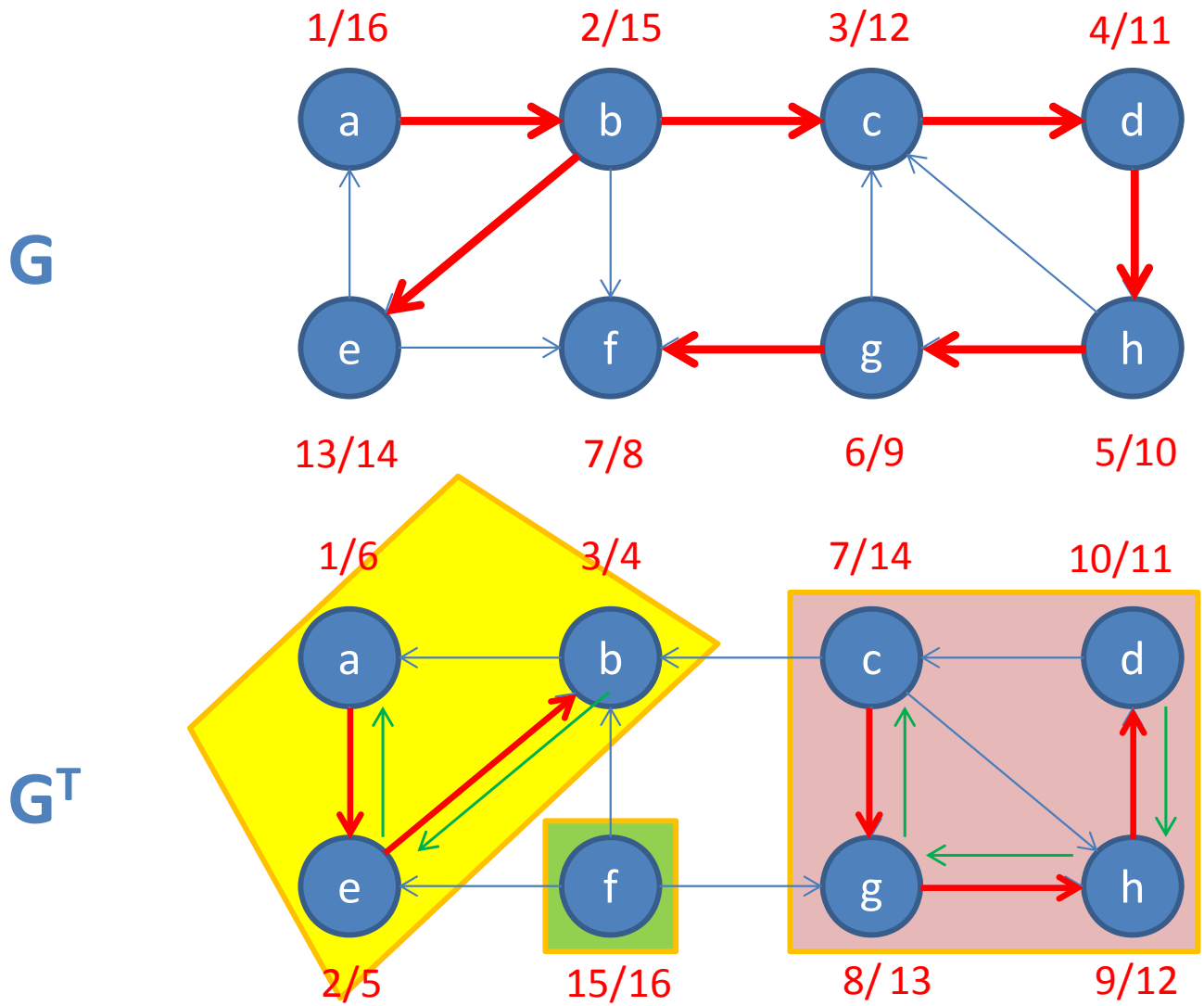
Proof:

- Suppose we have found all SCCs C_1, C2, …, C_{i-1} and consider C_i
- Then, $f(C_i) = f(u)$
- In G, if there exists edges from C to other components, all of them must be going out of C
- Hence, in G^T, there is no edges going out of C
- DFS(G^T, u) reports a DFS tree that covers all vertices in C





Example



"Infinite" number of MORE examples (but using Tarjan's algorithm) at:
http://www.comp.nus.edu.sg/~stevenha/visualization/dfsbfs.html

Summary

- Graph is big but a useful field in Computer Science
- Review of Graph DS & Graph Traversal, especially DFS
- Many graph problems are **NP-Complete**,
  but some variants (special cases) are in **P**
  - Including the all important 2-CNF-SAT problem for PA2 EF
- We focus on the discussion of Kosaraju's algorithm for finding Strongly Connected Components